

Smart Grid Energy Monitoring & Billing

Demo Video Script

Target length: 7–9 minutes

Project: EC8203 Applied Big Data Engineering

Architecture: Lambda architecture

Simulated day: 5 minutes

Before Recording

Start the project from the repository root:

```
docker compose up
```

Open these pages during the demonstration:

- FastAPI documentation: <http://localhost:8000/docs>
- Real-time grid load: <http://localhost:8000/realtime/grid-load>
- Renewable mix: <http://localhost:8000/realtime/renewable-mix>
- Daily billing report: <http://localhost:8000/billing/daily>
- Alerts: <http://localhost:8000/alerts>
- Metrics: <http://localhost:8000/metrics>
- Health check: <http://localhost:8000/healthz>
- Airflow: <http://localhost:8080>

Airflow login: admin / admin

Wait until the Docker services are running and the real-time grid endpoint returns zone data before starting the recording.

0:00–0:45 | Introduction

Show: README architecture diagram or project overview.

Say:

Hello. This is our Smart Grid Energy Monitoring and Billing pipeline.

>

The business problem is to give a utility company real-time visibility of electricity consumption and solar contribution, while also producing accurate daily household bills using tariff data.

>

The pipeline has two data sources. The first is a continuous smart-meter stream. The second is a daily batch file containing tariff and billing reference data.

>

The main outputs are real-time grid metrics, a daily household billing report, and alerts when the pipeline or renewable contribution falls below an expected level.

0:45–1:30 | Architecture Decision

Show: The Lambda architecture diagram.

Say:

We selected a Lambda architecture because this use case has two different processing requirements.

>

The speed layer provides low-latency grid metrics using Spark Structured Streaming. These metrics show current consumption and renewable contribution by grid zone.

>

The batch layer produces the authoritative daily bill. It waits for the daily tariff file, reprocesses the raw data for the simulated day, joins consumption with tariff information, and writes the final billing report.

>

We rejected a pure Kappa architecture because billing depends on a late-arriving reference file and must be correct rather than immediate. Lambda lets us use a fast path for monitoring and a separate correctness-focused path for billing.

1:30–2:15 | Starting the Pipeline

Show: Terminal running Docker Compose.

Say:

The complete environment is orchestrated using Docker Compose.

>

The stack includes Kafka for event ingestion, Spark Structured Streaming for the speed layer, PostgreSQL for storage, Airflow for batch orchestration, FastAPI for serving, and a health-check service for observability.

>

For this demonstration, one simulated day is compressed to five minutes instead of twenty-four hours. This allows the daily batch cycle to be demonstrated within a reasonable recording.

>

The smart-meter simulator sends readings every few seconds, while the batch simulator creates one tariff file per simulated day.

2:15–3:15 | Streaming Ingestion and Processing

Show: Meter-streamer and Spark logs, then open `/realtime/grid-load`.

Say:

This is the streaming ingestion path. Each event contains a meter ID, household ID, electricity consumption, solar generation, grid zone, and timestamp.

>

Kafka receives these events on the meter-readings topic. Spark Structured Streaming consumes the topic and groups readings into one-minute tumbling windows by grid zone.

>

Spark calculates total consumption, total solar generation, renewable share, active meters, and average load for each zone. The results are written to PostgreSQL and exposed through the API.

>

This endpoint shows the latest grid state by zone. The timestamp and values update as new smart-meter events are processed.

Refresh the endpoint.

The populated zones demonstrate that the speed layer is actively processing events rather than returning static data.

3:15–4:00 | Renewable Contribution

Show: `/realtime/renewable-mix`.

Say:

This endpoint aggregates the latest windows across all zones and reports the system-wide renewable share.

>

The utility can use this view to understand current grid load and how much of that demand is being offset by solar generation.

>

Because this is the speed layer, the result is designed for quick operational visibility. It is fresh and useful, but it can be recomputed if required.

4:00–5:00 | Daily Batch Billing

Show: Airflow at `http://localhost:8080` and open the `daily_billing_reconciliation` DAG.

Say:

The batch layer is orchestrated by Airflow. The DAG waits for the daily tariff file in the shared drop zone.

>

Once the file is available, the workflow validates its schema and row count. It then reads the raw meter archive for the simulated day, aggregates consumption per household, joins it with tariff and subsidy information, calculates the estimated bill, and stores the result in PostgreSQL.

>

This is the key difference between the two layers. The real-time dashboard can be slightly approximate and a few seconds old, but the billing report is recalculated as an authoritative daily result.

Open: `/billing/daily`.

This endpoint returns the consolidated household billing report, including consumption, solar generation, tariff data, subsidy status, and calculated bill amount.

5:00–5:45 | Storage and Serving

Show: FastAPI Swagger UI at `/docs`.

Say:

PostgreSQL is the queryable serving store for both processing paths.

>

The speed layer writes to the real-time metrics table, while the batch layer writes to the daily billing report table.

>

FastAPI exposes separate endpoint groups. The `/realtime` endpoints serve current grid metrics, the `/billing` endpoints serve authoritative daily reports, `/alerts` shows pipeline alerts, `/metrics` exposes Prometheus-compatible metrics, and `/healthz` provides a health probe.

>

This allows dashboards and other utility applications to consume the results without needing direct access to Kafka, Spark, or Airflow.

5:45–6:45 | Observability

Show: `/healthz`, `/metrics`, `/alerts`, and the health-check logs.

Say:

Observability is built into the pipeline rather than added only after a failure.

>

The health-check service continuously checks whether streaming data has arrived recently, whether the ingestion error rate is above its threshold, and whether renewable contribution is unusually low in a zone.

>

When a rule is triggered, the service writes a structured alert to PostgreSQL and publishes it to the `grid-alerts` Kafka topic.

>

The logs are structured JSON logs containing the timestamp, pipeline stage, component, and event details. This makes failures easier to search and diagnose.

>

The metrics endpoint exposes counters and gauges for processed records, ingestion quality, alerts raised, and overall pipeline health.

6:45–7:30 | Technology Justification

Show: README repository layout or architecture diagram.

Say:

Kafka was selected because it provides durable event ingestion and decouples the smart-meter source from the processing layer.

>

Spark Structured Streaming was selected because it supports event-time windowing and continuous meter-event processing.

>

Airflow manages the dependency between the daily file arriving and the billing job running.

>

PostgreSQL provides a queryable serving store that is appropriate for this demonstration.

>

FastAPI provides lightweight HTTP access to both real-time and batch results.

>

Docker Compose makes the complete system reproducible and allows all components to be demonstrated together.

7:30–8:15 | Limitations and Future Improvements

Say:

This is a single-node demonstration setup. In production, Kafka would use multiple brokers and replicated topics, while Spark would run on a distributed cluster.

>

The billing model is intentionally simplified and uses a flat tariff and subsidy calculation. A production system would support progressive tariff tiers, more complex regulations, and billing corrections.

>

The smart-meter and tariff data are simulated, and the simulated day is compressed to five minutes.

>

Production improvements would include stronger authentication, encrypted communication, persistent monitoring dashboards, alert routing to email or PagerDuty, and more extensive integration testing.

8:15–8:30 | Conclusion

Say:

To conclude, this project demonstrates an end-to-end Lambda architecture for smart-grid monitoring and billing.

>

It ingests streaming and batch data, processes them using separate speed and batch layers, stores the results in PostgreSQL, exposes them through FastAPI, orchestrates billing with Airflow, and monitors the pipeline using structured logs, metrics, and alerts.

>

The system answers both business questions: what is happening on the grid right now, and what should each household be billed once the daily tariff data is applied?

Quick Demo Checklist

- Docker Compose services are running.
- /healthz returns status ok.
- /realtime/grid-load returns multiple zones.
- /realtime/renewable-mix returns current totals.
- Airflow shows daily_billing_reconciliation.
- /billing/daily returns household records after a completed batch run.
- /alerts and /metrics are shown.
- Explain the five-minute simulated day.
- Explain why Lambda was selected instead of Kappa.